

PHISHGUARD

Research Integration and Documentation Guide

Final Research Report

Prepared by
PhishGuard Team

Florida International University
CIS 3950: Capstone 1
Professor Masoud Sadjadi

1. Purpose of the Guide

The purpose of the **PhishGuard Research Integration and Documentation Guide** is to organize the research completed throughout the semester and demonstrate how that work contributed to the project's development. This guide is not intended to repeat the earlier general research report. Instead, it brings the completed research into one structured document that shows how the project progressed from broad phishing awareness into explainable risk communication, prototype feature documentation, secure development planning, and practical URL-handling guidance.

The guide connects each research area to specific PhishGuard functions, development decisions, testing requirements, and project artifacts. Early research established the project's foundation through phishing indicators, user awareness, risk scoring, and safety guidance. Later research explored how scan warnings could be explained through the Sonar Analysis Log and the proposed Bait Breakdown feature. The research then advanced into secure prototype planning by addressing input validation, safe URL display, clear error messages, acceptance criteria, and quality-assurance support.

This document also records how the Research and Documentation contribution supported the prototype, final report, poster, presentation, and GitHub materials. It uses diagrams, workflows, and a research-to-feature matrix to show how research moved from general cybersecurity knowledge into practical project guidance. The guide will distinguish clearly between existing prototype functions and future improvements. For example, the Sonar Analysis Log is part of the current prototype documentation, while Bait Breakdown remains a planned enhancement intended to add plain-language explanations and recommended actions to detected warnings.

2. Background Research

PhishGuard is a cybersecurity-focused web application developed to help users recognize suspicious emails, text messages, and URLs through explainable phishing risk analysis and practical safety guidance. The semester research examined common phishing indicators, social engineering, user awareness, risk scoring, warning design, secure software development, and safe URL handling. These findings shaped the current prototype's focus on URL input, scan results, risk levels, the Sonar Analysis Log, recommended actions, and educational guidance. The research also supports Bait Breakdown as a proposed enhancement that would explain what was detected, why it may be risky, and what the user should do next. Together, these areas position PhishGuard as an awareness and decision-support tool rather than an enterprise-level detection system. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; Kehkashan et al.; Dawkins and Jacobs; OWASP Foundation, "Testing for Client-Side URL Redirect")

3. Research Organization and Topic Umbrella

The PhishGuard research can be organized into four connected areas: **Phishing Background and Awareness, Explainable Risk and User Guidance, Prototype Functions and Feature Documentation, and Secure Development and URL Handling**. These areas form a research umbrella that shows how the project developed throughout the semester. The research began by examining phishing threats and user-awareness needs, then expanded into the way scan results should be explained, how those findings could support prototype functions, and how suspicious

URLs should be handled safely. This organization demonstrates that the research did not remain a collection of separate cybersecurity topics. Each area contributed to specific prototype decisions, documentation, testing requirements, and final project artifacts.

3.1 Phishing Background and Awareness

The first research area establishes the cybersecurity foundation of PhishGuard. It includes common phishing indicators, suspicious emails and text messages, misleading URLs, urgency, social engineering, impersonation, requests for sensitive information, SMS phishing, and real-world attack examples. These topics explain the problem PhishGuard is intended to address and identify the warning signs that users may encounter before clicking a link or sharing private information. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; Anti-Phishing Working Group; Microsoft; Cloudflare)

The research also supports PhishGuard's educational purpose. The project is not intended only to produce a risk score. It should help users recognize why certain characteristics may indicate phishing and encourage safer decisions. Relevant warning categories include suspicious or shortened URLs, impersonation attempts, urgent language, requests for account or financial information, and other forms of social pressure. These findings support the project background, problem statement, warning-indicator selection, educational guidance, and poster motivation. (Federal Trade Commission; Marshall et al.)

3.2 Explainable Risk and User Guidance

The second area focuses on how PhishGuard communicates its findings. A user may understand that a URL received a high-risk result without understanding what caused that result. The explainable-risk research therefore supports warnings that identify what was detected, explain why it may be risky, and recommend a safe next action. (Roy et al.; Kazakova et al.; Buono et al.)

This area connects directly to the Sonar Analysis Log and the proposed Bait Breakdown feature. The Sonar Analysis Log displays the warnings detected during a scan, while Bait Breakdown would expand those warnings with plain-language explanations. For example, a URL-shortener warning could explain that shortened links may hide the actual destination. A missing-HTTPS warning could advise the user not to enter sensitive information. (Roy et al.; Kazakova et al.)

The research establishes a consistent explanation structure:

Warning title → What was detected → Why it may be risky → Recommended action

This format supports non-technical users, connects individual warnings to the overall risk result, and provides clearer guidance before users interact with suspicious content. Bait Breakdown remains a planned improvement to the existing Sonar Analysis Log rather than a confirmed implemented feature. (Kazakova et al.; Buono et al.)

3.3 Prototype Functions and Feature Documentation

The third research area shows how the cybersecurity findings were translated into documented prototype functions. The current prototype direction begins with a user entering a suspicious URL. The system then validates the input, examines visible indicators, presents a risk result, lists warnings in the Sonar Analysis Log, and provides a recommended action.

The documented prototype areas include:

- URL Input
- URL Validation and Scan
- Risk Score or Risk Level
- Sonar Analysis Log
- Recommended Action
- Cybersecurity Awareness Guidance
- Result Display
- Bait Breakdown as a proposed enhancement

This research area is important because it demonstrates that the semester’s research influenced actual project documentation. The findings supported feature descriptions, user stories, acceptance criteria, system diagrams, poster content, presentation material, and GitHub artifacts. It also provides consistent terminology that the team can use across the final deliverables.

3.4 Secure Development and URL Handling

The fourth area focuses on how PhishGuard should behave safely as a cybersecurity application. Because users may submit suspicious URLs, those URLs must be treated as untrusted input. The secure-development research supports validating URLs before analysis, rejecting empty or invalid input, avoiding automatic link opening, displaying suspicious URLs carefully, and providing clear error messages. (National Institute of Standards and Technology; OWASP Foundation, “Secure Coding Practices Checklist”; OWASP Foundation, “Input Validation Cheat Sheet”)

This area also connects the research to acceptance criteria and quality-assurance planning. Security should be included during prototype planning rather than added only after development. The team can create testable requirements for URL input, scan-result display, warning explanations, recommended actions, and error handling. (OWASP Foundation, “Application Security Verification Standard”)

The secure URL research further distinguishes the current project scope from possible future development. PhishGuard can remain realistic by using rule-based indicators and basic URL parsing during the current Capstone phase. Advanced services such as Google Safe Browsing may be documented as optional future enhancements rather than presented as current functionality. (MDN Web Docs; Google)

Together, the four research areas show clear progression:

Phishing threats and awareness → Explainable warnings and user guidance → Prototype functions and documentation → Secure development and URL handling

This organization provides the foundation for the sections that follow. The Research-to-Prototype Integration Map will show where each area influenced the project, while the Secure URL

Processing Workflow will demonstrate how the research supports the recommended behavior of the prototype.



Figure 1. PhishGuard Research Organization. The PhishGuard research is organized into four connected areas that support the project background, explainable scan results, prototype feature documentation, secure URL behavior, testing requirements, and final project deliverables.

4. Research-to-Prototype Integration Map

The PhishGuard research became valuable to the project when its findings were translated into development decisions, prototype functions, testing requirements, and final deliverables. The Research-to-Prototype Integration Map illustrates this relationship through four stages:

Research topic → Development decision → Prototype area → Project artifact

This structure demonstrates that the semester research did not remain separate from the technical and design work. Phishing-awareness research helped determine what the application should examine. Explainability research influenced how scan results should be communicated. Secure-development research guided how submitted URLs should be validated and displayed. The resulting decisions then supported prototype descriptions, user stories, acceptance criteria, QA planning, the final report, the poster, the presentation, and GitHub documentation.

4.1 Phishing Indicators and URL Analysis

The foundational phishing research identified warning signs such as suspicious links, misleading or shortened URLs, impersonation, urgent language, requests for sensitive information, and redirect behavior. These findings helped define which visible indicators PhishGuard should consider during an analysis. Although the original project concept included suspicious emails, text messages, and URLs, the current prototype focus was narrowed to suspicious URL analysis to keep the project manageable during the Capstone phase. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; Anti-Phishing Working Group; Microsoft; Cloudflare)

This research influenced the documented URL Scan function. The scan is intended to examine characteristics such as missing HTTPS, suspicious keywords, excessive URL length, URL shorteners, and possible redirect behavior. The purpose is not to declare that every URL with one of these characteristics is automatically malicious. Instead, the indicators help the system identify reasons for caution and communicate those reasons to the user. (OWASP Foundation, “Testing for Client-Side URL Redirect”; MDN Web Docs)

The path of integration for this research can therefore be represented as:

Phishing indicators → Define visible warning categories → URL Scan and Sonar Analysis Log → Prototype documentation, poster content, and risk-analysis guidance

The same findings also support the educational purpose of PhishGuard. Warning indicators can be reused in cybersecurity tips, demonstration examples, and user-facing explanations so users can recognize similar phishing techniques outside the application. (Marshall et al.; Federal Trade Commission)

4.2 Explainable Warnings and the Sonar Analysis Log

The explainable-risk research established that a risk score alone may not give users enough information to make a safe decision. Users should be able to see what PhishGuard detected, understand why the finding may be concerning, and receive guidance about what to do next. This research directly supports the Sonar Analysis Log, which displays the individual warnings associated with a scan rather than presenting only a general result. (Roy et al.; Kazakova et al.; Buono et al.)

The research also produced a consistent model for warning explanations:

Warning title → What was detected → Why it may be risky → Recommended action

This structure converts technical findings into information that can be understood by users without advanced cybersecurity knowledge. It can also keep the wording consistent across the interface, feature documentation, user stories, and acceptance criteria. (Kazakova et al.; Buono et al.)

The corresponding integration path is:

Explainable-warning research → Connect results to understandable reasons → Sonar Analysis Log and Result Display → Feature descriptions, interface text, acceptance criteria, and final report

This pathway represents one of the central purposes of PhishGuard. The project is not intended to function as a black-box scanner that produces an unexplained label. Its value comes from making the warning signs visible and helping users interpret them.

4.3 Bait Breakdown and User Guidance

Bait Breakdown developed from the explainable-warning research as a proposed improvement to the Sonar Analysis Log. The current prototype documentation identifies the Sonar Analysis Log as the area that displays detected warnings. Bait Breakdown would expand this area by adding a short explanation and recommended action for each warning. (Roy et al.; Kazakova et al.)

For example, if the system identifies a URL shortener, Bait Breakdown could explain that shortened links may conceal the actual destination. If the system detects missing HTTPS, it could advise the user to avoid entering passwords, payment details, or other sensitive information. The feature would therefore connect warning detection to practical decision-making rather than leaving the user with an unexplained technical label. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; OWASP Foundation, “Testing for Client-Side URL Redirect”)

The integration path is:

Explainable-risk and warning-design research → Provide context for each detected warning → Proposed Bait Breakdown feature → User story, acceptance criteria, backlog documentation, and future prototype improvement

Bait Breakdown must be described carefully throughout the final report. It is a **research-supported proposed enhancement**, not a confirmed current implementation. The current prototype already includes or documents the Sonar Analysis Log, while Bait Breakdown represents the next step for making that log more informative. The team must also decide whether explanations should appear directly under each warning or inside expandable sections.

4.4 Risk Scoring and Result Presentation

The risk-score research supports presenting scan findings in a form that users can interpret quickly. A risk level can summarize the analysis, but the score should remain connected to the warning indicators that produced it. The documented Risk Score function therefore works

together with the Sonar Analysis Log rather than replacing it. The score provides the summary, while the log provides the supporting details. (Dawkins and Jacobs; Kehkashan et al.)

The Result Display function brings these areas together by presenting the risk level, detected warnings, available explanations, and recommended action in one organized result. This structure supports PhishGuard's explainability-first direction because it gives users both a quick assessment and the evidence needed to understand it.

The integration path is:

Risk-analysis research → Connect risk levels to detected indicators → Risk Score and Result Display → Prototype interface, demonstration materials, poster features, and QA criteria

This relationship also creates an important consistency requirement. Warning labels used during scanning should match the labels displayed in the Sonar Analysis Log, any Bait Breakdown explanation, and the final risk result. Otherwise, users may not understand how the result was produced.

4.5 User-Safety Research and Recommended Actions

Research from the semester documents repeatedly supports giving users a practical next step after a suspicious result. PhishGuard may advise users not to open the link, not to enter sensitive information, to verify the sender or destination, or to access the organization through an official website instead. (Federal Trade Commission; Cybersecurity and Infrastructure Security Agency)

These findings influenced the Recommended Action function and Cybersecurity Awareness Guidance. The Recommended Action function responds to the immediate scan result, while the awareness guidance helps users recognize similar risks in future messages or links.

The integration path is:

User-safety and awareness research → Provide clear next-step guidance → Recommended Action and Awareness Guidance → Interface content, educational resources, poster material, and presentation support

This pathway reinforces that PhishGuard is both an analysis tool and an awareness resource. The intended outcome is not simply that users receive a warning, but that they understand how to respond safely.

4.6 Secure Development and URL Handling

The secure-development research influenced how the prototype should accept and process suspicious URLs. Because the application receives potentially unsafe input, each submitted URL should be treated as untrusted. The prototype should validate its format, detect empty or invalid entries, avoid automatically opening the link, and display it in a way that does not encourage unsafe interaction. (National Institute of Standards and Technology; OWASP Foundation, "Secure Coding Practices Checklist"; OWASP Foundation, "Input Validation Cheat Sheet")

These findings support the URL Input function, URL-validation behavior, error messages, safe result display, and QA testing. They also helped convert general security goals into testable requirements. For example, the application can be tested to confirm that invalid input produces a

clear message, HTTP URLs can be identified, and suspicious links remain non-clickable or are otherwise displayed safely. (OWASP Foundation, “Application Security Verification Standard”)

The integration path is:

Secure-development and URL-handling research → Treat submitted URLs as untrusted input → URL Input, validation, error handling, and safe display → Acceptance criteria, QA cases, sprint planning, and security documentation

The research also establishes an important scope boundary. More advanced reputation checks, automatic page fetching, server-side URL requests, or integrations such as Google Safe Browsing remain possible future enhancements. They should not be presented as current prototype functionality unless the team later implements and verifies them. (OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”; Google)

4.7 Integration with Final Project Deliverables

The research influenced more than the prototype interface. It also provided support for the team’s project artifacts. The function documentation connects prototype areas to research sources, user stories, acceptance criteria, and risk considerations. The poster outline uses research to explain the project problem, goal, current prototype focus, Sonar Analysis Log, proposed Bait Breakdown feature, and future work.

The final integration path is therefore:

Completed research → Organized project knowledge → Documentation and artifact support → Final report, diagrams, poster, presentation, GitHub repository, user stories, and QA materials

Together, these pathways demonstrate the practical contribution of Research and Documentation to PhishGuard. The research established what the application should recognize, how results should be explained, how submitted URLs should be handled, and how those findings could be converted into testable and presentable project artifacts.

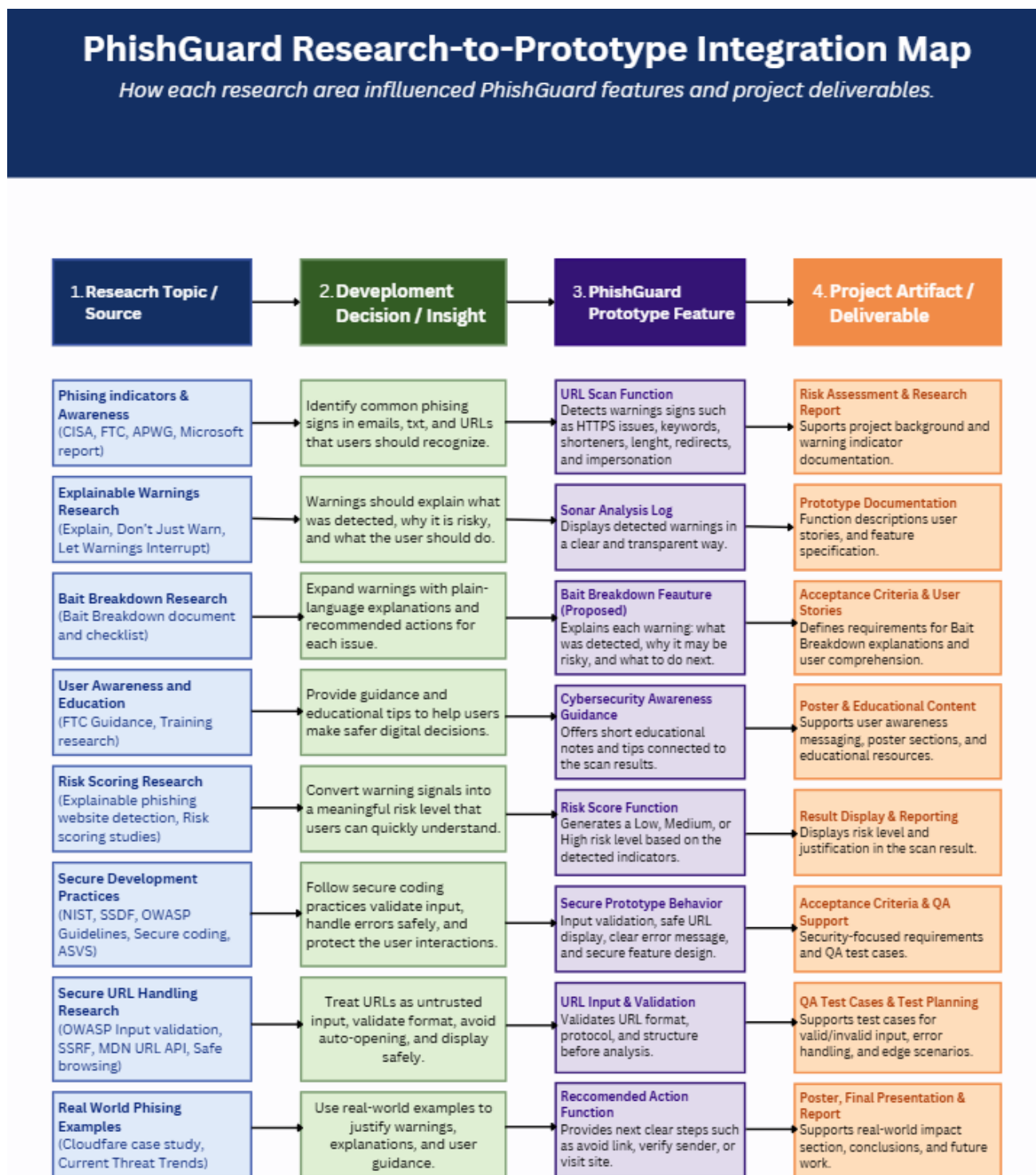


Figure 2. PhishGuard Research-to-Prototype Integration Map. The diagram shows how the semester's phishing-awareness, explainability, user-guidance, and secure-development research informed prototype decisions, documented functions, testing requirements, and final project deliverables. Current prototype areas are distinguished from proposed enhancements such as Bait Breakdown and future advanced URL threat-checking.

5. Secure URL Processing Workflow

The Secure URL Processing Workflow translates the semester’s research into a recommended sequence for how PhishGuard should respond when a user submits a suspicious URL. Unlike the previous section, which mapped research topics to prototype features and deliverables, this section focuses on the behavior of the prototype itself. It explains how a URL should move from user input through validation, indicator analysis, risk presentation, warning explanation, and recommended action.

The workflow is a planning and documentation model rather than completed implementation code. Its purpose is to give the development, design, and QA teams a shared description of how the prototype should behave. Because PhishGuard accepts potentially suspicious links, every submitted URL must be treated as untrusted input. The application should validate the input, avoid automatically opening it, display it carefully, and explain the results in a way that supports safer user decisions. (OWASP Foundation, “Input Validation Cheat Sheet”; OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”; MDN Web Docs)

The complete recommended workflow is:

User → Enter URL → Validate URL → Valid? → Analyze Indicators → Risk Score → Sonar Analysis Log → Bait Breakdown → Recommended Action → Safe User Decision

When the URL is missing or invalid, the workflow follows a separate path:

Invalid Input → Display Clear Error Message → Return to URL Input

5.1 URL Entry

The workflow begins when the user pastes or types a suspicious link into the URL Input field. This step should remain simple and direct, so users understand exactly where to submit the link for review. The system should accept the URL as text and should not open or follow the link when it is entered.

The URL Input area is also a security-sensitive boundary because it introduces user-controlled information into the prototype. The application must not assume that the submitted content is trustworthy. It should first determine whether the input is present and whether it resembles a valid URL before beginning any analysis. (OWASP Foundation, “Input Validation Cheat Sheet”; OWASP Foundation, “Secure Coding Practices Checklist”)

At this stage, the interface should provide clear instructions such as:

Enter the full URL you would like PhishGuard to review.

An example can help the user understand the expected format:

`https://example.com`

The entry field should not transform the URL into a clickable link. Treating it as plain text reduces the chance that the user will accidentally open the suspicious destination while attempting to analyze it. (OWASP Foundation, “Testing for Client-Side URL Redirect”)

5.2 URL Validation

After the URL is submitted, PhishGuard should validate the input before displaying a scan result. Validation should first check whether the field is empty. If no URL was entered, the application should stop the workflow and ask the user to provide one. (OWASP Foundation, “Input Validation Cheat Sheet”)

The system should then check whether the submitted value can be interpreted as a URL and whether it uses an expected protocol such as HTTP or HTTPS. Basic URL parsing may also identify components such as the protocol, hostname, path, and search parameters. If the input cannot be parsed, the application should display a clear, non-technical error message instead of continuing as though the entry were valid. (MDN Web Docs)

For example:

Empty input

Please enter a URL before beginning the scan.

Invalid input

Please enter a valid URL, such as `https://example.com`.

An HTTP address may still pass basic format validation, but the absence of HTTPS can be recorded as a warning indicator. The secure URL plan therefore distinguishes between an invalid URL that cannot continue and a validly formatted URL that may still contain suspicious characteristics. (MDN Web Docs)

5.3 Validation Decision

The **Valid?** decision point separates error handling from the analysis process.

When the input is invalid, PhishGuard should:

1. Stop the scan.
2. Display a clear explanation of the problem.
3. Keep the suspicious input from being opened or processed as trustworthy.
4. Return the user to the URL Input field.
5. Allow the user to correct and resubmit the URL.

When the input is valid, the application can continue to basic URL analysis.

This decision point is important because PhishGuard should not produce a risk score for input that the system cannot correctly interpret. Continuing after failed validation could confuse the user, produce misleading results, or create unsafe behavior. The secure-development research therefore supports validation before results, clear error states, and consistent handling of missing or unsupported input. (OWASP Foundation, “Input Validation Cheat Sheet”; OWASP Foundation, “Application Security Verification Standard”)

5.4 Indicator Analysis

Once the URL passes validation, PhishGuard can analyze visible characteristics associated with phishing risk. The semester documents identify possible warning indicators including: (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; OWASP Foundation, “Testing for Client-Side URL Redirect”)

- Missing HTTPS
- Suspicious keywords
- URL shorteners
- Excessive or complex URL length
- Suspicious redirects
- Unknown or unusual domain patterns
- Requests for sensitive information
- Possible impersonation attempts

The current project documentation supports a realistic, rule-based prototype direction. These checks are intended to identify visible warning characteristics and support user awareness. They do not prove that a website is malicious, nor should the prototype claim enterprise-grade detection based only on these indicators. (Kehkashan et al.; Dawkins and Jacobs)

For example, a shortened URL may hide the destination, but shortened links are not automatically phishing. Missing HTTPS can be a reason for caution, especially when sensitive information is requested, but it is not conclusive evidence by itself. The prototype should therefore present these findings as warning indicators that contribute to a broader risk result rather than as absolute declarations. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission)

The system should also avoid automatically fetching the website, previewing its content, or following redirects during the current prototype workflow. More advanced server-side or API-assisted analysis would create additional security and implementation requirements that remain outside the confirmed current scope. (OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”)

5.5 Risk Score or Risk Level

After examining the selected indicators, PhishGuard should summarize the result through a risk score or risk level. The purpose of this stage is to give the user a quick interpretation of the scan without forcing them to evaluate every technical detail independently. (Dawkins and Jacobs; Kehkashan et al.)

The result may use categories such as:

- Low Risk
- Moderate Risk

- High Risk

However, the score should never appear without supporting information. The semester research emphasizes that users should understand which detected characteristics contributed to the result. The risk level therefore works together with the Sonar Analysis Log rather than replacing it. (Dawkins and Jacobs; Kehkashan et al.)

A result might appear as:

Risk Level: High

Multiple suspicious indicators were detected. Review the Sonar Analysis Log before interacting with this URL.

The final score should remain connected to consistent warning labels. If the scoring process identifies “URL Shortener Detected,” the same wording should appear in the Sonar Analysis Log and any corresponding explanation. This consistency helps the user understand how the result was produced.

5.6 Sonar Analysis Log

The Sonar Analysis Log presents the specific warning indicators detected during the scan. Its role is to make the analysis transparent by showing users what PhishGuard found instead of presenting only a general risk label. (Roy et al.; Kazakova et al.)

Possible entries include:

- Missing HTTPS
- URL Shortener Detected
- Suspicious Keyword Found
- Excessive URL Length
- Suspicious Redirect
- Unusual Domain Pattern

The warning names should be concise, consistent, and visibly connected to the final risk level. The secure URL plan also recommends making warning labels and the risk result more visually prominent than the submitted link itself. Suspicious URLs should be shown carefully, preferably as plain text rather than as prominent clickable content. (OWASP Foundation, “Testing for Client-Side URL Redirect”)

The Sonar Analysis Log represents part of the current documented prototype direction. It establishes the foundation for explainable results because it reveals the warning signs that contributed to the scan.

5.7 Bait Breakdown Explanations

Bait Breakdown represents the explanation stage of the workflow. It is a proposed enhancement that would expand the Sonar Analysis Log by explaining each warning in plain language. It should not be described as confirmed current functionality unless the team verifies that it has been implemented. (Roy et al.; Kazakova et al.; Buono et al.)

For each warning, Bait Breakdown should answer three questions:

1. What was detected?
2. Why may it be risky?
3. What should the user do next?

For example:

URL Shortener Detected

What was detected:

The submitted URL appears to use a shortened link.

Why it may be risky:

Shortened links can conceal the final website destination, making it more difficult to determine where the link will lead.

Recommended action:

Do not open the link unless the sender and destination can be verified.

This format turns a technical warning into understandable guidance. The research supports keeping explanations concise, using non-technical language, and connecting each explanation to the scan result or risk score. (Kazakova et al.; Buono et al.)

The team still needs to decide whether these explanations should appear directly beneath each Sonar Analysis Log entry or inside expandable sections. Either approach can support the workflow as long as the warning title remains visible and the explanation is easy to access.

5.8 Recommended Action

Following the warning explanation, PhishGuard should provide at least one practical safety recommendation. The recommended action should match the warning indicators found during the scan rather than relying on the same generic message for every result. (Federal Trade Commission; Cybersecurity and Infrastructure Security Agency)

Possible recommendations include:

- Do not open the suspicious link.
- Do not enter passwords, payment information, or other sensitive data.
- Verify the sender through a trusted contact method.
- Visit the organization's official website directly.
- Preview or verify the destination of a shortened URL.
- Treat unclear redirects with caution.

The Recommended Action function is essential because identifying risk without explaining how to respond leaves the user with an incomplete result. This stage converts the analysis into safer digital behavior.

5.9 Safe User Decision

The final stage is not a technical system output but the intended outcome of the workflow: a safer and more informed user decision.

After reviewing the risk level, Sonar Analysis Log, available warning explanations, and recommended action, the user should have enough information to decide whether to avoid the link, verify its source, visit an official site directly, or seek additional information.

This outcome reflects the central purpose of PhishGuard. The project is not intended to make every decision for the user or guarantee that a URL is safe. It is designed to help users pause, understand suspicious characteristics, and respond more carefully before clicking links or sharing sensitive information.

5.10 Safe Display Throughout the Workflow

Safe URL display applies across the entire process rather than at only one stage. PhishGuard should never automatically open the submitted URL. It should avoid presenting a suspicious link as trustworthy and should not make the link more visually prominent than the warning results. (OWASP Foundation, “Testing for Client-Side URL Redirect”; OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”)

The submitted URL should be displayed as plain text when possible. If future versions add page previews, server-side requests, redirect following, or reputation-service integrations, those features should be treated as higher-risk additions requiring separate security planning. (OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”)

The current workflow can remain practical for the Capstone prototype by relying on basic validation, rule-based indicators, explainable warnings, and user guidance. Google Safe Browsing or similar external services may be documented as future options, but they should not be presented as current capabilities unless the team confirms their implementation. (Google)

5.11 Connection to Acceptance Criteria and QA

The workflow gives the team a direct basis for acceptance criteria and testing. Each stage can be evaluated through observable behavior. (OWASP Foundation, “Application Security Verification Standard”)

The principal workflow requirements are:

- The user can enter a URL for review.
- Empty input produces a clear error message.
- Invalid URL formatting produces a clear error message.
- Valid HTTP and HTTPS URLs can continue to analysis.
- Missing HTTPS can be displayed as a warning.
- The application does not automatically open submitted URLs.
- Detected warnings appear in the Sonar Analysis Log.

- The result includes a risk level.
- The result includes at least one recommended action.
- Bait Breakdown explanations are included only when that proposed feature is implemented.
- Suspicious URLs are displayed safely.

The secure URL plan also provides example QA cases for blank input, invalid text, HTTP and HTTPS addresses, shortened URLs, suspicious redirects, and safe display. These tests allow the workflow to become measurable project behavior rather than remaining only a conceptual diagram. (OWASP Foundation, “Input Validation Cheat Sheet”; MDN Web Docs)

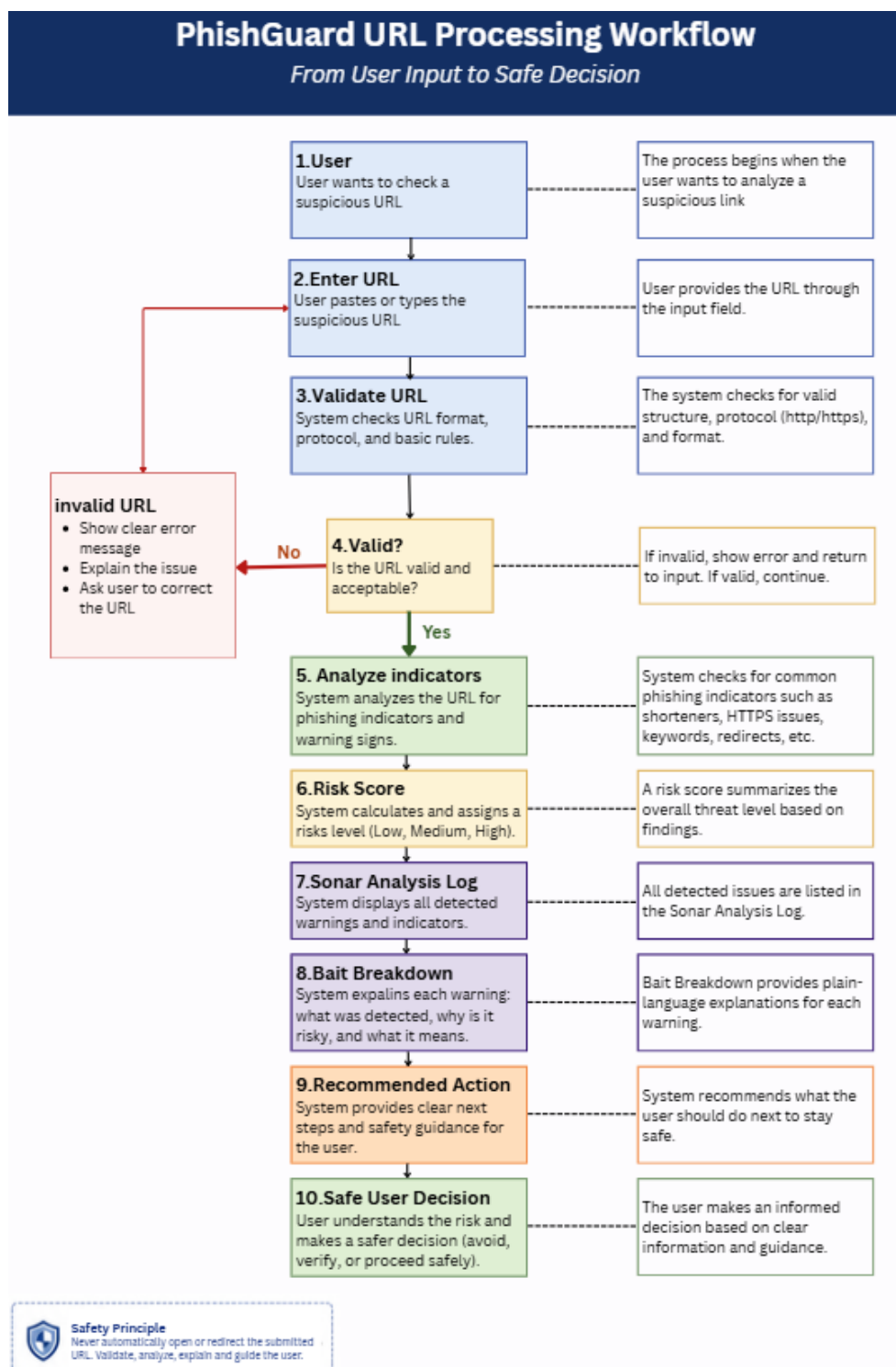


Figure 3. PhishGuard Secure URL Processing Workflow. The workflow shows the recommended path from URL submission through validation, indicator analysis, risk presentation, warning explanation, recommended action, and safer user decision-making. Invalid input follows a separate error path that returns the user to URL entry. Bait Breakdown is shown as a proposed explanation layer, while advanced URL fetching and external threat-checking remain future possibilities rather than confirmed current functions.

6. Research Evolution Timeline

The Research Evolution Timeline documents how the PhishGuard research and documentation developed throughout the semester. Sections 3 through 5 explained how the research is organized, how it supports the prototype, and how the proposed URL-processing workflow should operate. This section instead focuses on progress: how the work moved from broad cybersecurity investigation into feature planning, secure-development guidance, and final project support.

The timeline should not be interpreted as a software-release history or as evidence that every researched feature was implemented. It represents the progression of the Research and Documentation contribution. Each milestone built upon the previous work rather than replacing it, creating the following sequence:

Research foundation → Explainable-risk planning → Feature documentation → Secure prototype guidance → Final project integration

The nine milestones used in the timeline are Project Research Begins, Final Research Report, Explainable Risk Research, Bait Breakdown, Prototype Function Descriptions, Secure Prototype Development, Secure URL Handling, Poster Support, and Research Integration Guide. This order reflects the structure previously established for Diagram 4.

6.1 Project Research Begins

The research process began when the team identified the need for a cybersecurity awareness tool that could help users evaluate suspicious digital content. The earliest project direction considered suspicious emails, text messages, and URLs, with emphasis on phishing indicators, user-awareness needs, explainable analysis, and safer digital decision-making.

At this stage, the main purpose of the research was to establish the problem PhishGuard would address. The team needed to understand which phishing risks users commonly encounter, why users may struggle to recognize those risks, and how the project could remain useful without attempting to become an enterprise-level security product.

Timeline description:

The team identified the need for PhishGuard and began researching phishing threats, user-awareness needs, and explainable risk analysis.

6.2 Final Research Report

The Final Research Report organized the initial evidence base into a broad foundation for PhishGuard. It examined common phishing warning signs, user-safety guidance, current phishing trends, explainable detection, risk scoring, secure URL handling, awareness training, and real-world phishing examples. The report also proposed risk categories and connected the findings to possible application features, acceptance criteria, and team responsibilities.

This milestone established that PhishGuard should do more than label content as suspicious. The project should identify warning signs, explain the reason for concern, and provide users with a safer next action. It also established the broad research foundation from which the later, more specialized documents developed.

Timeline description:

Comprehensive research established the project foundation through phishing indicators, awareness, trends, risk scoring, user guidance, and secure URL considerations.

6.3 Explainable Risk Research

After establishing the general phishing foundation, the research narrowed toward how warnings should be presented to users. This phase examined the limitations of generic security alerts and emphasized that warnings should provide context, understandable reasoning, and practical guidance. (Roy et al.; Kazakova et al.; Buono et al.)

The research supported a consistent explanation model in which each warning communicates what was detected, why it may be risky, and what action the user should take. It also considered warning placement and whether explanations should appear directly beneath a warning or through expandable content. These decisions were important because a warning is useful only when users notice it, understand it, and know how to respond. (Roy et al.; Kazakova et al.; Buono et al.)

Timeline description:

Research focused on explaining phishing warnings clearly, interpreting risk results, and helping users understand why a URL may be suspicious.

6.4 Bait Breakdown

The Bait Breakdown concept developed as a direct application of the explainable-risk research. It was designed as a proposed enhancement to the Sonar Analysis Log, which already displays the warning indicators detected during a scan. Bait Breakdown would expand those indicators with plain-language explanations and recommended user actions.

This phase transformed the general principle of “explain the warning” into a defined feature concept. The documentation identified the user problem, related phishing indicators, explanation format, connection to the Risk Score, user story, and acceptance criteria. It also preserved an important scope distinction: Bait Breakdown remained a planned feature improvement rather than a confirmed implemented function.

Timeline description:

The Bait Breakdown concept expanded explainable warnings into plain-language descriptions, reasons for risk, and recommended actions.

6.5 Prototype Function Descriptions

The next stage translated the research into specific prototype functions. The documentation described URL Input, URL Scan, Risk Score, Sonar Analysis Log, Recommended Action, Cybersecurity Awareness Guidance, Result Display, and Documentation and Artifact Support. Bait Breakdown was also described as the proposed explanation layer that could expand the Sonar Analysis Log.

This milestone demonstrated that the research had moved beyond theoretical cybersecurity discussion. The findings were converted into descriptions that could support design decisions, user stories, acceptance criteria, prototype development, QA planning, the poster, and the final presentation.

Timeline description:

Research findings were translated into detailed descriptions of prototype functions, including URL input, scanning, risk presentation, warning logs, recommended actions, and educational guidance.

6.6 Secure Prototype Development

Once the feature direction had been documented, the research shifted toward how the prototype should be planned safely. This phase connected NIST and OWASP guidance to input validation, safe URL handling, error messages, result display, acceptance criteria, and QA requirements. (National Institute of Standards and Technology; OWASP Foundation, “Secure Coding Practices Checklist”; OWASP Foundation, “Application Security Verification Standard”)

The secure-development research established that security should be considered during planning rather than added only after implementation. Because PhishGuard accepts suspicious URLs, the URL Input function must be treated as a security-sensitive boundary. Submitted URLs should be considered untrusted, validated before analysis, displayed carefully, and prevented from opening automatically. The research also helped turn broad security goals into testable requirements. (National Institute of Standards and Technology; OWASP Foundation, “Secure Coding Practices Checklist”; OWASP Foundation, “Input Validation Cheat Sheet”)

Timeline description:

Secure-development practices were researched to guide input validation, safe interaction, error handling, acceptance criteria, and QA planning.

6.7 Secure URL Handling

The Secure URL Handling and Validation Plan narrowed the security research further by focusing specifically on user-submitted links. It documented recommended behavior for empty input, invalid formats, HTTP and HTTPS protocols, URL parsing, safe link display, redirects, shortened URLs, the Sonar Analysis Log, Bait Breakdown explanations, and recommended user actions. (OWASP Foundation, “Input Validation Cheat Sheet”; OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”; MDN Web Docs; Google)

This stage converted secure-development principles into a practical workflow and a set of QA test cases. It also clarified that the current prototype should avoid automatically fetching, previewing, or following submitted URLs. More advanced services or server-side checking were treated as future possibilities rather than confirmed current functionality. (OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”; Google)

The document therefore represented a major transition from general research to direct development planning. It gave the team practical guidance for sprint planning, prototype behavior, acceptance criteria, QA testing, feature documentation, and future artifacts.

Timeline description:

Specific research translated secure-development guidance into URL validation, safe display, redirect handling, workflow planning, acceptance criteria, and QA cases.

6.8 Poster Support

As the project approached its final deliverables, the research and feature documentation were reorganized for presentation. The Poster Content Outline identified the project overview, problem, goal, prototype focus, Sonar Analysis Log, proposed Bait Breakdown feature, supporting research areas, Agile process, team contributions, and future work.

This milestone shows that the research was not produced only for internal reference. It became part of the team's explanation of what PhishGuard is, why the project matters, how the prototype works, and how each team role contributed to its development.

Timeline description:

Research and documentation were organized to support the project poster, including the project problem, research areas, key features, team contributions, and future work.

6.9 Research Integration Guide

The final milestone is the current Research Integration and Documentation Guide. This guide consolidates the completed research, explains how it developed into specialized topics, and connects those topics to prototype functions, security decisions, testing requirements, and final artifacts.

Unlike the earlier documents, which each focused on a specific research question or feature area, this guide presents the full documentation history as one connected project contribution. It brings together the topic umbrella, Research-to-Prototype Integration Map, Secure URL Processing Workflow, Research Evolution Timeline, Research-to-Feature Support Matrix, future recommendations, and conclusion.

Timeline description:

All research, documentation, and project artifacts were integrated into one guide showing how the Research and Documentation work supported the prototype and final deliverables.

6.10 Overall Progression

The timeline demonstrates that the documentation gradually matured through five broader stages:

Research → Feature planning → Prototype planning → Development guidance → Final deliverables

The first stage established the phishing and awareness foundation. Explainable-risk research then turned general findings into user-facing warning principles. Bait Breakdown and the function descriptions converted those principles into feature documentation. Secure-development and URL-handling research transformed the features into safer, testable prototype guidance. Finally, the poster outline and Research Integration Guide organized the work for presentation, evaluation, GitHub documentation, and future project development.

This progression demonstrates that the research effort did more than collect information from cybersecurity sources. It continuously adapted to the needs of the project. Each document responded to a more specific question than the one before it:

- What phishing problems should PhishGuard address?

- How should suspicious findings be explained?
- Which functions should represent those findings?
- How should those functions be planned securely?
- How should the work be tested, presented, and preserved?

The timeline therefore provides evidence of both the project's development process and the contribution of the Research and Documentation role. It can support the final report, poster, presentation, GitHub repository, Sprint Review, and instructor evaluation.

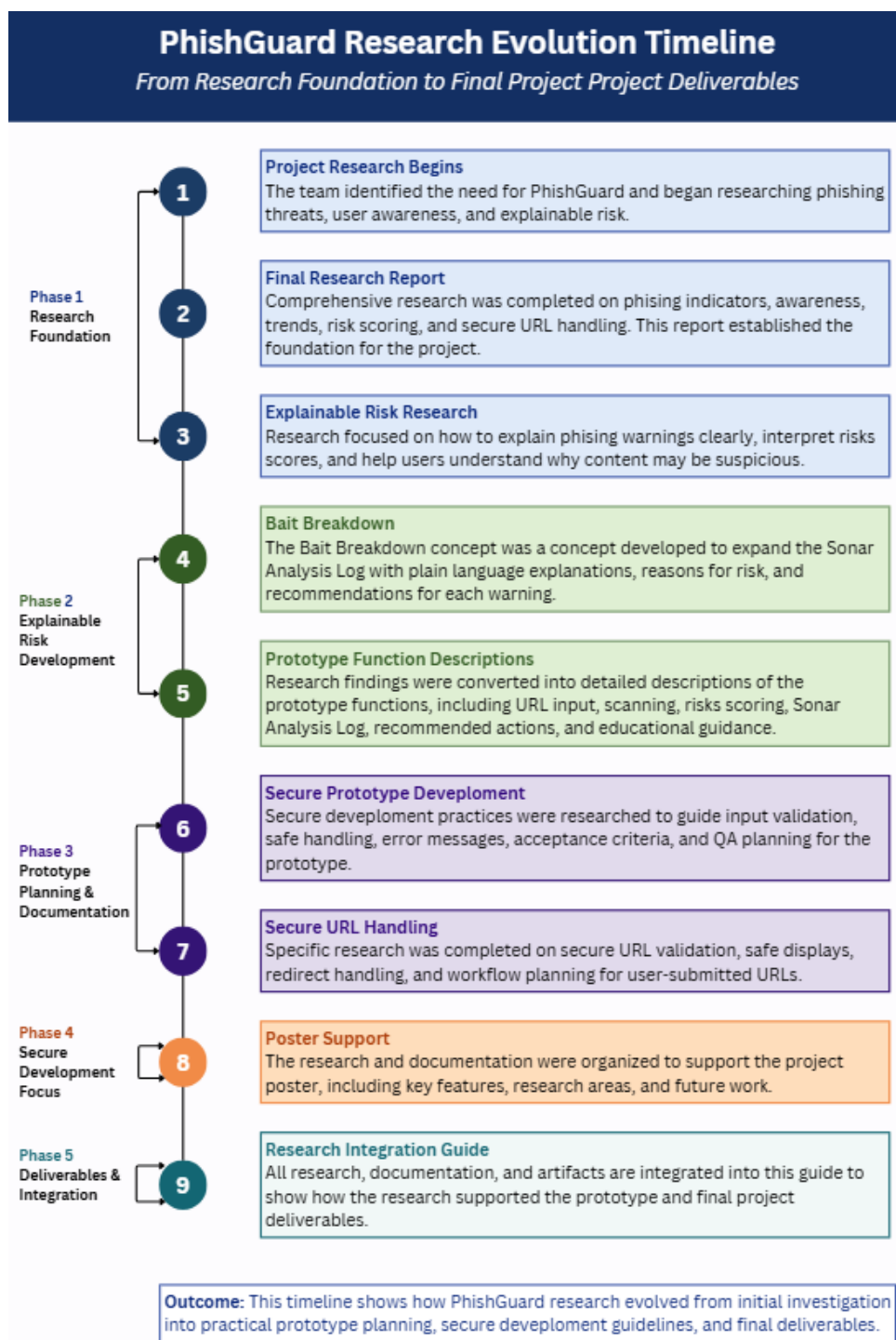


Figure 4. PhishGuard Research Evolution Timeline. The timeline documents how the semester's work progressed from initial phishing research into explainable-risk planning, the proposed Bait Breakdown feature, prototype function documentation, secure-development guidance, URL-handling requirements, poster support, and the final Research Integration and Documentation Guide.

7. Research-to-Feature Support Matrix

The Research-to-Feature Support Matrix summarizes how the semester’s major research areas influenced PhishGuard’s documented prototype features and development direction. It provides a concise reference connecting the research foundation to its practical application in the project. The matrix also distinguishes current prototype areas from proposed and future enhancements.

Research Area	Supported PhishGuard Feature or Area	Research Contribution
Phishing indicators and suspicious URL characteristics	URL Scan	Defines warning signs such as missing HTTPS, suspicious keywords, URL shorteners, long URLs, redirects, and unusual domain patterns. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; OWASP Foundation, “Testing for Client-Side URL Redirect”)
Risk interpretation	Risk Score or Risk Level	Summarizes the warning indicators in a clear form that helps users understand the overall scan result. (Dawkins and Jacobs; Kehkashan et al.)
Explainable phishing warnings	Sonar Analysis Log	Shows the specific warnings that contributed to the Risk Score instead of providing an unexplained result. (Roy et al.; Kazakova et al.)
Contextual warning explanations	Bait Breakdown — Proposed	Expands each warning by explaining what was detected, why it may be risky, and what action the user should take. (Roy et al.; Kazakova et al.; Buono et al.)
User-safety guidance	Recommended Action	Gives users practical next steps, such as avoiding the link, verifying the sender, or visiting the official website directly. (Federal Trade Commission; Cybersecurity and Infrastructure Security Agency)
Cybersecurity awareness	Educational Guidance	Helps users understand common phishing tactics and recognize similar risks in future messages or URLs. (Marshall et al.; Federal Trade Commission)
Secure software development	URL Input and Validation	Supports treating submitted URLs as untrusted input, checking their format, and rejecting empty or invalid entries. (National Institute of Standards and Technology; OWASP Foundation, “Input Validation Cheat Sheet”)
Safe interface behavior	Safe URL and Result Display	Prevents automatic link opening and avoids presenting suspicious URLs as trustworthy or encouraging users to click them. (OWASP Foundation, “Testing for Client-Side URL Redirect”; OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”)
Security requirements and testing	Acceptance Criteria and QA Tests	Converts secure-development research into testable requirements for validation, warnings, errors, and result behavior. (OWASP Foundation, “Application Security Verification Standard”)
Advanced threat checking	External URL Reputation Service — Future	Identifies API-based reputation checking as a possible future enhancement rather than current prototype functionality. (Google)

The matrix shows that the prototype features developed from several connected research areas. Phishing-indicator research supports the URL Scan by identifying the characteristics that should produce warnings. The Risk Score summarizes those warnings, while the Sonar Analysis Log displays the individual findings that contributed to the result. (Cybersecurity and Infrastructure Security Agency; Dawkins and Jacobs; Roy et al.)

Explainable-risk research supports Bait Breakdown as a proposed expansion of the Sonar Analysis Log. The feature would provide a warning title, an explanation of what was detected, a reason the finding may be risky, and a recommended action. It remains a proposed enhancement and should not be presented as confirmed current functionality. (Roy et al.; Kazakova et al.; Buono et al.)

Secure-development research supports the URL Input, validation, error-handling, and safe-display areas. Submitted URLs should be treated as untrusted input, checked before analysis, prevented from opening automatically, and displayed without encouraging unsafe interaction. (National Institute of Standards and Technology; OWASP Foundation, “Secure Coding Practices Checklist”; OWASP Foundation, “Input Validation Cheat Sheet”)

The same research provides a basis for acceptance criteria and QA testing. The prototype can be evaluated to confirm that it detects blank or invalid input, shows understandable error messages, identifies selected warning indicators, displays results safely, and provides appropriate user guidance. (OWASP Foundation, “Application Security Verification Standard”)

8. Recommendations and Future Improvements

The recommendations for PhishGuard should be approached in stages. The current prototype direction focuses on suspicious URL analysis through URL Input, URL Scan, Risk Score, Sonar Analysis Log, Recommended Action, and educational guidance. Bait Breakdown remains a proposed improvement, while advanced URL reputation checking and broader phishing-analysis capabilities remain future possibilities. These distinctions should remain clear, so the project does not present planned ideas as completed functionality.

Future development should first improve the clarity, consistency, and safety of the existing prototype before introducing more complex detection methods. This staged approach allows the research to continue guiding development without expanding the project beyond what can be tested and documented responsibly.

8.1 Short-Term Improvements

The priority should be refining the current URL-analysis experience. These improvements build directly on the semester’s completed research and can strengthen the prototype without changing its central scope.

Refine Bait Breakdown.

The team should finalize the format and placement of Bait Breakdown explanations. Each warning should consistently identify what was detected, explain why it may be risky, and provide a recommended user action. The team should also decide whether these explanations will appear directly beneath each Sonar Analysis Log warning or inside expandable sections. The explainable-risk research supports Bait Breakdown as a practical improvement that could make scan results more understandable without requiring a more advanced detection system. (Roy et al.; Kazakova et al.; Buono et al.)

Improve the Sonar Analysis Log.

Warning labels should be consistent across the URL Scan, Risk Score, Sonar Analysis Log, and Bait Breakdown. For example, a warning identified during analysis as “URL Shortener Detected” should use the same wording everywhere it appears. The log should also make it clear which findings contributed to the final risk result.

Strengthen user-interface wording.

Instructions, warning descriptions, error messages, and recommended actions should use clear language for non-technical users. The interface should explain what information the user must

enter, what happened during the scan, and what the result means. Invalid input should produce a specific and understandable message rather than a generic error.

Expand educational guidance.

PhishGuard should provide short educational notes connected to the scan result. These notes can explain common tactics such as shortened URLs, suspicious redirects, fake login pages, impersonation, urgency, and requests for sensitive information. This would reinforce the project's goal of helping users learn from the analysis rather than only receiving a score. (Marshall et al.; Federal Trade Commission)

Complete repeatable QA testing.

Testing should use the documented cases for empty input, invalid formats, HTTP and HTTPS addresses, shortened URLs, suspicious redirects, and safe URL display. The team should also test whether warning labels, risk levels, explanations, and recommended actions remain consistent throughout the result. This converts the semester's research into measurable prototype behavior. (OWASP Foundation, "Input Validation Cheat Sheet"; MDN Web Docs; OWASP Foundation, "Application Security Verification Standard")

8.2 Medium-Term Improvements

After the current workflow is stable and tested, PhishGuard can expand the depth of its URL analysis.

Expand phishing-indicator detection.

The prototype can add more rule-based checks for suspicious domain patterns, misleading subdomains, unusual URL structures, encoded characters, impersonation attempts, and sensitive-information requests. Any new indicator should include a clear explanation and should not be treated as conclusive evidence by itself. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; OWASP Foundation, "Testing for Client-Side URL Redirect")

Improve risk-scoring logic.

The current Risk Score should be supported by documented rules describing how warning indicators affect the result. More serious findings may receive greater weight than lower-risk characteristics. However, the score should remain explainable, and every major contribution to the result should appear in the Sonar Analysis Log. (Dawkins and Jacobs; Kehkashan et al.)

Provide more detailed reporting.

Future results could organize findings into categories such as link structure, connection security, redirect behavior, domain characteristics, and social-engineering indicators. This could give users more detail while preserving a clear overall risk level and recommended action.

Evaluate an external URL reputation service.

Google Safe Browsing or a similar service could eventually supplement the prototype's rule-based analysis by checking whether a URL appears in a database of known unsafe resources. This should remain optional future functionality until the team documents privacy considerations, permissions, implementation limits, error behavior, and the effect of external-service failures. The current prototype does not need an external API to remain realistic for the Capstone scope. (Google)

Clarify the technical analysis model.

Before more advanced URL checking is added, the team should document whether analysis occurs in the browser, on a server, through an external API, or through a combination of these approaches. Automatic page fetching, preview generation, and redirect following create additional security concerns and should not be added without a separate design and testing process. (OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet”)

8.3 Long-Term Improvements

Long-term improvements represent possible directions for a later version of PhishGuard. They are not current requirements and would require additional research, development, security review, and testing.

Expand beyond URL-only analysis.

The broader project concept includes suspicious emails, text messages, and URLs. A future version could allow users to submit message content for analysis of urgency, impersonation, sensitive-information requests, misleading links, and other social-engineering indicators. The present prototype should continue to describe URL analysis as its current focus. (Anti-Phishing Working Group; Cloudflare; Microsoft)

Explore machine-learning-assisted classification.

Machine learning could eventually help identify more complex phishing patterns. However, such a feature would require appropriate training data, performance evaluation, false-positive and false-negative analysis, privacy safeguards, and explanations for how classifications are produced. It should support not replace the project’s explainability goals.

Develop browser-extension support.

A future browser extension could allow users to review links while browsing. This would require careful permission design, clear user consent, secure communication with any PhishGuard service, and safeguards against collecting unnecessary browsing information.

Expand cybersecurity-awareness resources.

The educational side of PhishGuard could grow into a larger awareness section containing examples, interactive explanations, common phishing scenarios, quizzes, and updated safety guidance. This would strengthen the project’s role as both a scanning tool and an educational resource.

Use continuously updated threat intelligence.

A future production version could incorporate updated threat information, so its analysis reflects new phishing campaigns and malicious domains. This would require a reliable data source, updating procedures, privacy planning, and clear communication about the limits and freshness of the information.

These long-term directions were identified as possible future improvements in the approved report plan, along with browser-extension support, email scanning, expanded awareness resources, and continuous threat-intelligence updates. They should be treated as exploratory recommendations rather than promised features.

8.4 Recommended Development Priority

PhishGuard should not attempt to implement every proposed improvement at once. The most appropriate order is:

Current prototype quality → Explainable results → Stronger rule-based analysis → Optional external services → Broader long-term capabilities

The immediate focus should remain on stable URL input, reliable validation, consistent warning labels, clear risk presentation, safe URL display, understandable explanations, recommended actions, and repeatable testing. Once these areas are complete, the team can evaluate whether more advanced integrations or broader content-analysis features are appropriate.

This staged direction preserves the realistic scope of the Capstone prototype while demonstrating that the semester's research can continue supporting future development. It also ensures that new features remain aligned with PhishGuard's primary purpose: helping users recognize suspicious content, understand why it may be risky, and make safer digital decisions.

9. Conclusion

The PhishGuard Research Integration and Documentation Guide demonstrates how the project's cybersecurity research developed from a broad investigation of phishing threats into practical support for prototype planning, feature documentation, secure development, quality assurance, and final project deliverables. The research began by establishing the project's foundation through common phishing indicators, suspicious URLs, social-engineering methods, user-awareness needs, risk interpretation, and safer digital behavior. It then became progressively focused as the team examined explainable warnings, prototype functions, secure URL handling, and testable requirements. (Cybersecurity and Infrastructure Security Agency; Federal Trade Commission; Anti-Phishing Working Group; Microsoft)

A central contribution of the research was defining PhishGuard as an explainable awareness and decision-support tool. The project is intended to do more than produce a risk label. The Risk Score provides users with a summary of the analysis, while the Sonar Analysis Log identifies the warnings that contributed to that result. Research into explainable phishing warnings further established that users should be told what was detected, why the finding may be risky, and what action they should take. This direction connects the technical scan results to user education, transparency, and safer decision-making. (Roy et al.; Kazakova et al.; Buono et al.; Dawkins and Jacobs)

The research also supported Bait Breakdown as a proposed improvement to the Sonar Analysis Log. This feature would expand warning labels with plain-language explanations and recommended actions for findings such as missing HTTPS, shortened URLs, suspicious keywords, redirects, and excessive URL length. Bait Breakdown remains a proposed enhancement rather than a confirmed current implementation. Maintaining this distinction ensures that the final documentation represents the prototype accurately while still showing how the research can guide future development. (Roy et al.; Kazakova et al.; Buono et al.)

Secure-development research strengthened the project by examining how PhishGuard itself should handle suspicious content. Submitted URLs should be treated as untrusted input, validated before analysis, displayed carefully, and prevented from opening automatically. These

findings supported clearer error handling, safe result presentation, acceptance criteria, and QA test cases for empty input, invalid formats, HTTP and HTTPS links, shortened URLs, redirects, and non-clickable display. The research therefore moved beyond explaining phishing risks and helped define how the prototype should behave responsibly. (National Institute of Standards and Technology; OWASP Foundation, “Secure Coding Practices Checklist”; OWASP Foundation, “Input Validation Cheat Sheet”)

Together, the research areas supported the documented PhishGuard workflow. URL Input receives the suspicious link, URL Scan examines visible warning indicators, Risk Score summarizes the findings, and the Sonar Analysis Log shows the reasons behind the result. Recommended Action and Cybersecurity Awareness Guidance then help the user decide what to do and recognize similar risks in the future. Bait Breakdown represents a possible explanation layer, while advanced reputation services and API-assisted threat checking remain future options rather than current requirements. (OWASP Foundation, “Input Validation Cheat Sheet”; MDN Web Docs; Federal Trade Commission)

The completed documentation also contributed to the project beyond the prototype. Research findings were translated into user stories, acceptance criteria, QA planning, workflow diagrams, feature descriptions, sprint documentation, poster content, presentation support, GitHub materials, and the final report. This traceability demonstrates that the Research and Documentation role did not simply collect cybersecurity information. It helped the team define why features were included, how they should behave, how they could be tested, and how the project should be presented.

The Research Integration and Documentation Guide brings these contributions together in a single reference. The Research Organization and Topic Umbrella establishes the major research categories. The Research-to-Prototype Integration Map shows how those categories affected development decisions and deliverables. The Secure URL Processing Workflow explains the recommended prototype behavior. The Research Evolution Timeline records how the work progressed throughout the semester. The Research-to-Feature Support Matrix provides a direct connection between the research and the documented prototype areas. Finally, the recommendations identify how PhishGuard can continue improving without overstating the current scope.

Overall, the PhishGuard research provided the cybersecurity foundation, explainability principles, secure-development guidance, testing support, and educational direction required to shape the project. By organizing the semester’s work into a cohesive guide, the team now has a reusable resource that supports the current Capstone submission and future development. The result reflects PhishGuard’s primary purpose: helping users recognize suspicious content, understand why it may be risky, and make safer digital decisions.

Works Cited

- Anti-Phishing Working Group. *Phishing Activity Trends Report, 4th Quarter 2025*. APWG, 18 Feb. 2026, docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf.
- Buono, Paolo, Giuseppe Desolda, Francesco Greco, and Antonio Piccinno. “Let Warnings Interrupt the Interaction and Explain: Designing and Evaluating Phishing Email Warnings.” *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems*, Association for Computing Machinery, 2023, doi.org/10.1145/3544549.3585802.
- Cloudflare. “The Mechanics of a Sophisticated Phishing Scam and How We Stopped It.” *The Cloudflare Blog*, 9 Aug. 2022, blog.cloudflare.com/2022-07-sms-phishing-attacks/.
- Cybersecurity and Infrastructure Security Agency. “Recognize and Report Phishing.” *CISA*, www.cisa.gov/secure-our-world/recognize-and-report-phishing.
- Dawkins, Shanee, and Jody Jacobs. *NIST Phish Scale User Guide*. National Institute of Standards and Technology, 15 Nov. 2023, doi.org/10.6028/NIST.TN.2276.
- Federal Trade Commission. “How To Recognize and Avoid Phishing Scams.” *Consumer Advice*, Federal Trade Commission, consumer.ftc.gov/articles/how-recognize-avoid-phishing-scams.
- Google. “Google Safe Browsing.” *Google for Developers*, developers.google.com/safe-browsing.

Kazakova, Vera A., Jena D. Hwang, Bonnie J. Dorr, Yorick Wilks, J. Blake Gage, Alex Memory, and Mark A. Clark. “SPLAIN: Augmenting Cybersecurity Warnings with Reasons and Data.” **arXiv**, 2023, arxiv.org/abs/2311.11215.

Kehkashan, Tanzila, et al. “Explainable Phishing Website Detection for Secure and Sustainable Cyber Infrastructure.” **Scientific Reports**, vol. 15, article 41751, 2025, doi.org/10.1038/s41598-025-27984-w.

Marshall, Nina, Daniel Sturman, and Jaime C. Auton. “Exploring the Evidence for Email Phishing Training: A Scoping Review.” **Computers & Security**, vol. 139, 2024, article 103695, doi.org/10.1016/j.cose.2023.103695.

MDN Web Docs. “URL: URL() Constructor.” **Mozilla**, developer.mozilla.org/en-US/docs/Web/API/URL/URL.

Microsoft. **Microsoft Digital Defense Report 2025**. Microsoft, 2025, [www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/](<http://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/>).

National Institute of Standards and Technology. **Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities**. NIST Special Publication 800-218, Feb. 2022, csrc.nist.gov/pubs/sp/800/218/final.

OWASP Foundation. “Application Security Verification Standard.” *OWASP*, owasp.org/www-project-application-security-verification-standard/.

OWASP Foundation. “Input Validation Cheat Sheet.” *OWASP Cheat Sheet Series*, cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html.

OWASP Foundation. “OWASP Top Ten Web Application Security Risks.” *OWASP*, owasp.org/www-project-top-ten/.

OWASP Foundation. “Secure Coding Practices Checklist.” *OWASP Secure Coding Practices Quick Reference Guide*, owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en/02-checklist/.

OWASP Foundation. “Server Side Request Forgery Prevention Cheat Sheet.” *OWASP Cheat Sheet Series*, cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html.

OWASP Foundation. “Testing for Client-Side URL Redirect.” *OWASP Web Security Testing Guide*, OWASP Foundation, owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/11-Client-side_Testing/04-Testing_for_Client-side_URL_Redirect.

Roy, Sayak Saha, Cesar Torres, and Shirin Nilizadeh. ““Explain, Don’t Just Warn!’: A Real-Time Framework for Generating Phishing Warnings with Contextual Cues.” *arXiv*, 2025, arxiv.org/abs/2505.06836.